Name: Jacinto Jimenez
Class: CS 79C Section 1753
Instructor: Vicky Seno
Date: October 27, 2026

Final Project: AWS Architecture Portfolio Project

Project: Incident Handoff Tracker | Deployment: AWS us-west-2 (Oregon)

Documentation status: Deployment and live demonstration verified; teardown and signed incident-route test not yet documented. This report should be updated after final cleanup.

Executive Summary

I designed and deployed a low-cost incident handoff service for fictional technical support records. API Gateway provides the HTTPS interface; AWS Lambda processes each request; DynamoDB stores searchable incident records; Amazon S3 holds JSON snapshots; and Amazon CloudWatch records executions and monitors errors. One CloudFormation YAML file creates the complete architecture, while the public health endpoint and backend tests supply reproducible evidence of a working deployment.

Problem, Audience, and Demonstrated Workflow

Short incident handoffs can become fragmented across temporary notes and messages. This portfolio application standardizes a fictional record with a UUID, title, summary, severity, created timestamp, and author so that another technician can retrieve it by identifier. It is a teaching prototype for analysts and architecture reviewers—not an operational ticketing system and not a location for company or customer data.

Real AWS Evidence Obtained

Verification	Captured result
Infrastructure	CloudFormation incident-handoff-portfolio: CREATE_COMPLETE (Oct. 27, 2026).
Public API	HTTPS GET /health returned status ok, student Jacinto Jimenez, region us-west-2.
Create incident	Lambda console event for POST /incidents returned statusCode 201, Demo conveyor alert.
Data persistence	DynamoDB Scan returned the same incident; S3 contained its UUID-named JSON file.
Retrieve incident	Lambda console event for GET /incidents/{id} returned statusCode 200.
Observability	CloudWatch log stream held START/END/REPORT; Lambda Errors alarm was OK.
Cost estimate	AWS Pricing Calculator: $0.46/month and $5.52/year (projected workload).

The evidence screenshots are included in Appendix B and in the public GitHub evidence/ folder. Their status values are real results rather than sample forms. A signed request through the IAM-protected incident routes and an AWS cleanup confirmation are not yet evidenced.

Architecture: Components, Data Flow, and Alternatives

Figure 1 (Appendix A) diagrams the integrated architecture. GET /health is public so a reviewer can view the project status. Protected POST /incidents and GET /incidents/{incidentId} require API Gateway AWS_IAM authorization. A successful POST writes a record to DynamoDB, then uploads a JSON archive to S3. All Lambda invocations send logs and error metrics to CloudWatch.

Selected service	Function and explicit alternative rejected
API Gateway HTTP API	HTTPS routing, Lambda proxy integration, IAM auth, and stage throttling. Chosen over an always-running ALB to avoid a load balancer for a very small service.
AWS Lambda (Python)	Validates JSON, generates UUID, reads/writes records. Chosen over EC2 to avoid managing a VM and idle instance costs.
DynamoDB On-Demand	Stores items keyed by incidentId. Chosen over RDS because there is no relational join requirement or fixed database instance.
Amazon S3 Standard	Private JSON archive objects. Chosen over EC2 local disk for durable, encrypted object storage independent of the compute runtime.
Amazon CloudWatch	Native logs and Lambda Errors alarm. Chosen over self-managed monitoring software that would require a separate server.
CloudFormation / IAM	One versioned YAML file provisions resources; scoped IAM grants access. Chosen over local Terraform setup for a single-account AWS lab and over embedded access keys.

Infrastructure as Code

The repository contains template.yaml (deployable), infra/template.base.yaml and src/handler.py. The stack creates a DynamoDB PAY_PER_REQUEST table; S3 bucket with block-public-access, SSE-S3 encryption and seven-day incident-object expiration; Lambda Python 3.12 on arm64 with 128 MB memory and a 10-second timeout; IAM execution policy; API Gateway HTTP routes and throttling of two requests per second (burst five); seven-day Lambda log retention; and a standard Lambda Errors alarm. CloudFormation Outputs expose the health URL, API base URL, table, bucket, function, and alarm names.

The service composition demonstrates more than four working AWS components. The API health check verified API Gateway-to-Lambda. The Lambda console POST and GET tests verified its backend data workflow; an IAM-signed HTTP request remains a separate planned test, not a recorded accomplishment.

Implementation and Testing

I uploaded the CloudFormation template in the AWS Console, used stack name incident-handoff-portfolio, retained the StudentName parameter, acknowledged creation of IAM resources, and verified CREATE_COMPLETE in Oregon. The stack Outputs supplied a public HealthURL that returned the expected project and student information. The deployment used the web console because AWS CloudShell was unavailable earlier during account verification.

Next, I executed a Lambda console event simulating POST /incidents. It returned HTTP-style statusCode 201 and the UUID ec99fa8c-95c3-4c42-8670-a061fc7a4cb6. I scanned the DynamoDB table and found one matching record, then opened the private S3 incidents/ path and found the UUID-named JSON object. A second simulated GET invocation returned statusCode 200 and the same incident. CloudWatch Logs and the Lambda Errors alarm verified basic monitoring.

Observed Challenges and Solutions

- CloudShell could not initially create an environment; I used the CloudFormation web interface to avoid blocking deployment.
- The first DynamoDB Pricing Calculator estimate was incorrectly set to 5,000 million operations/month. Changing reads and writes to 0.005 million/month reduced the cost estimate to a representative small-workload figure.
- The actual Lambda function was deployed as arm64, while Lambda was modeled as x86 in the calculator. That mismatch should be corrected for a precise comparative estimate; the saved calculator total remains a labeled forecast.

Security

Incident routes use AWS_IAM authorization; the handler uses an IAM execution role instead of static keys. IAM permissions are scoped to DynamoDB PutItem/GetItem on the project table, S3 PutObject under incidents/, and its Lambda log group. HTTPS secures transit, DynamoDB provides default encryption at rest, and S3 is SSE-S3 encrypted with public access blocked. Unlike an EC2 application, this design has no user-managed inbound security groups. The public health route deliberately reveals no incident data. Limitations include no Cognito end-user identity, WAF, multi-tenant authorization, or verified signed client test.

Reliability and Disaster Recovery

Managed regional API Gateway, Lambda, DynamoDB and S3 remove the need to maintain one critical VM. A local Availability Zone issue does not depend on one self-hosted application server, although a Region-wide issue can still interrupt this single-Region design. Planned recovery objectives are RTO of four hours and RPO of one hour; these are targets only and were not tested. DynamoDB PITR, replication and a restoration drill are not included. If DynamoDB writes succeed but the following S3 upload fails, the archive may be incomplete; a queue or stream-backed retry mechanism would mitigate this.

Performance and Cost

Lambda scales on demand and the DynamoDB table uses on-demand request billing; API Gateway stage throttling controls burst traffic. The tested POST invocation completed in 3,091.98 ms with 98 MB maximum memory, while the GET completed in 2,416.57 ms using 92 MB, both configured at 128 MB. The GET execution showed 267.29 ms initialization. These are two observed console invocations, not p95 response latency or a load test. Main bottlenecks are cold starts, sequential DynamoDB/S3 writes, and the intentionally low API throttle.

The AWS Pricing Calculator estimate in Appendix B is a hypothetical monthly scenario, not an invoice: 10,000 HTTP API requests; 10,000 Lambda invocations at 128 MB and 2,500 ms each; 5,000 DynamoDB reads and 5,000 writes, 1 GB table storage; 1 GB S3 Standard storage with 5,000 PUT and GET requests; and 0.1 GB of logs plus one standard alarm. It totals $0.46/month ($5.52/year): API Gateway $0.01, Lambda $0.00 under a free-tier assumption, DynamoDB $0.25, S3 $0.05, and CloudWatch $0.15. The calculator modeled x86 while the deployed template uses arm64; update the configuration before relying on platform-specific compute comparisons. Actual bills depend on account usage, eligibility and other services.

Two clear cost optimizations are (1) using HTTP API, Lambda and DynamoDB on-demand in place of always-on EC2/RDS/ALB/NAT infrastructure and (2) bounding log/archive retention to seven days for this temporary fictional-data workload.

AWS Well-Architected Self-Review

Pillar	Strength	Honest weakness / improvement
Security	IAM-protected incident routes, encrypted private storage, narrowly scoped role.	Add Cognito/JWT identities, WAF rules and a signed HTTP integration test.
Reliability	AWS-managed regional services and S3 snapshots for successful writes.	Use asynchronous archive retries, DynamoDB PITR and tested disaster recovery.
Cost Optimization	No permanently running EC2; pay-per-request database; short retention.	Track actual billing, align calculator with arm64, review alarm/log expenses.

What I Would Do Differently

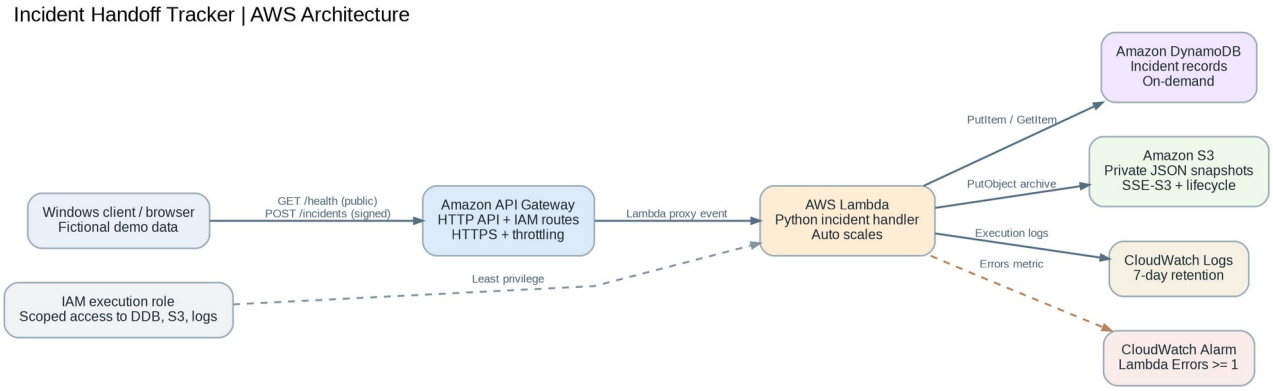
With more time, I would use DynamoDB Streams or SQS to decouple incident creation from S3 archiving, adding retries and a dead-letter path. I would also test the IAM-protected API from an authorized workstation, add authenticated end users and a simple interface, run controlled load tests, and document a verified recovery drill before treating the design as production-ready.

Deployment and Cleanup Status

The attached screenshots prove the deployment and successful functional checks. IAM-signed end-to-end incident API testing and CloudFormation DELETE_COMPLETE have not yet been captured. The live project should be torn down after evidence collection: empty the S3 bucket, delete the stack, confirm resources are gone, and update this report/README to state the observed result.

Appendix A — AWS Architecture Diagram

Figure 1. Original portfolio diagram committed to the public repository. Solid arrows represent application data flow; dashed arrows represent permissions, logs, or monitoring.

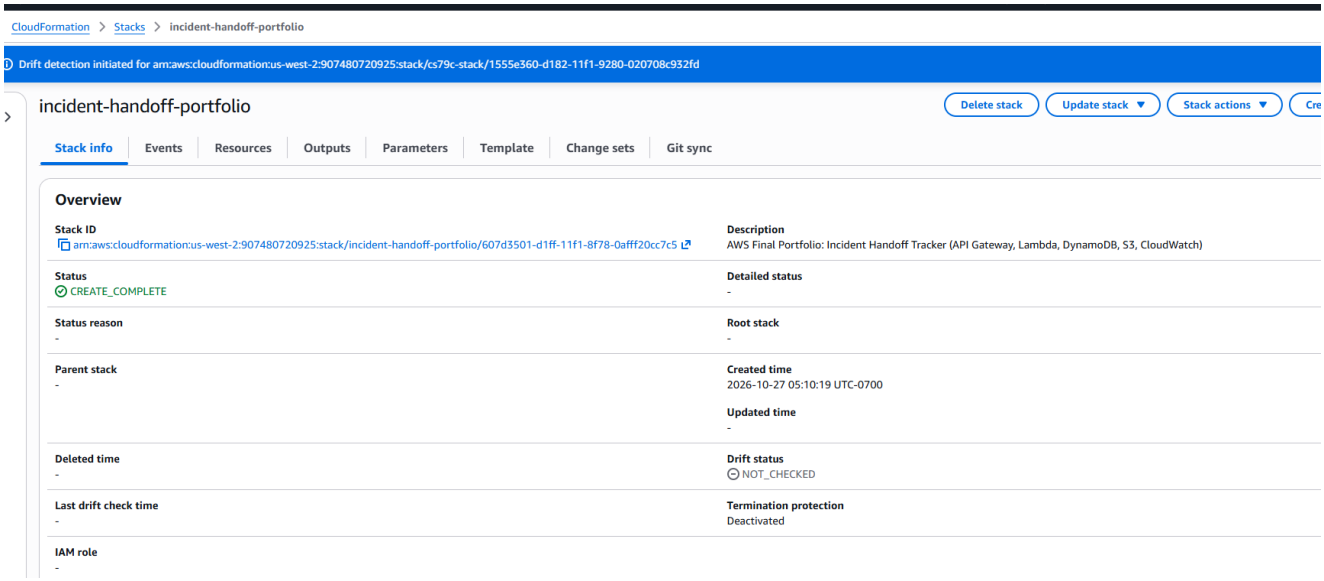


Components are provisioned from CloudFormation template.yaml. The diagram is also embedded in the repository README.

Appendix B — Live AWS Screenshots

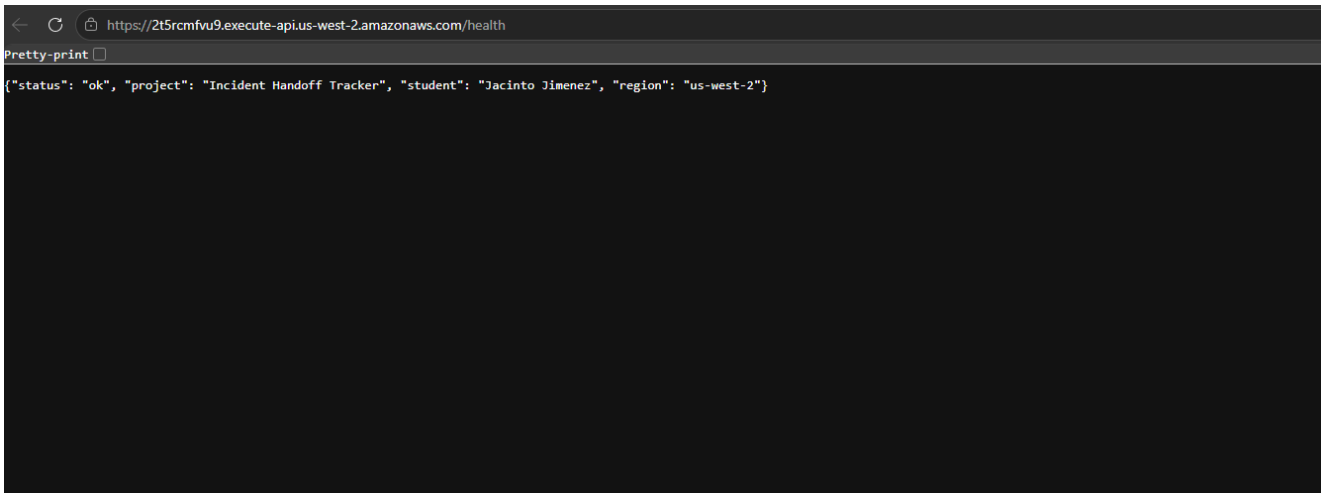
All evidence shown below is from user-captured AWS screens, not mockups. Personal browser chrome was cropped when needed; reported AWS results were preserved.

Screenshot 1 — CloudFormation CREATE_COMPLETE



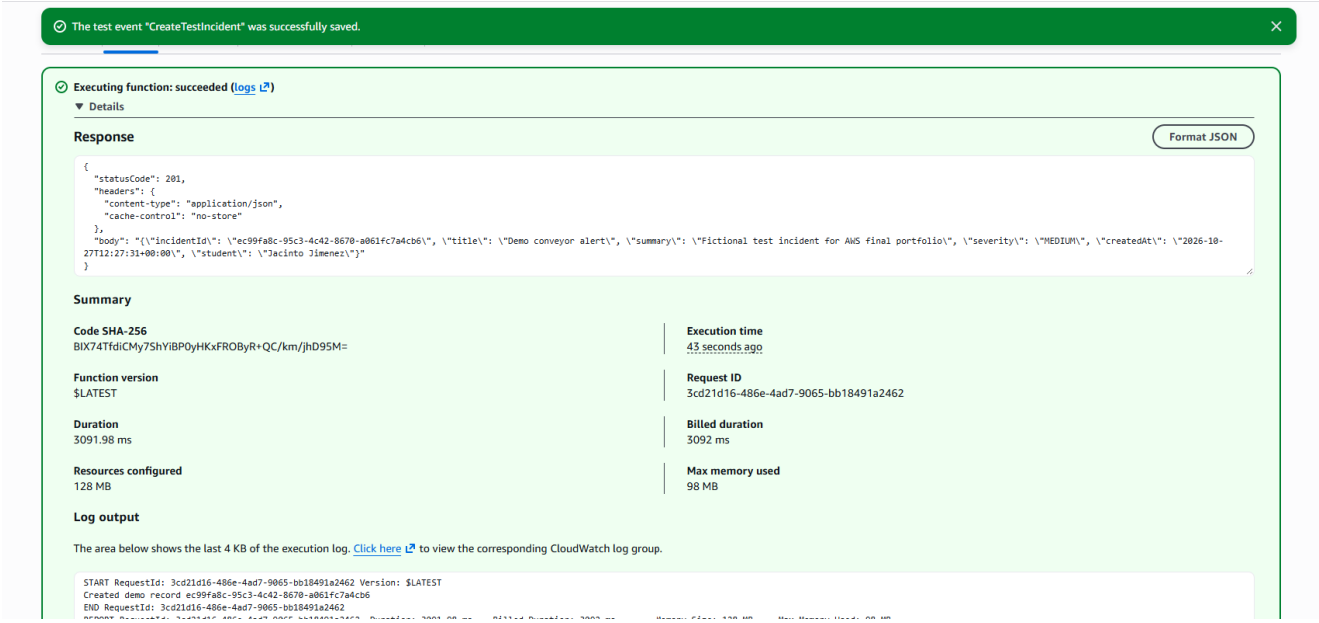
Stack deployed successfully in us-west-2.

Screenshot 2 — Public Health API



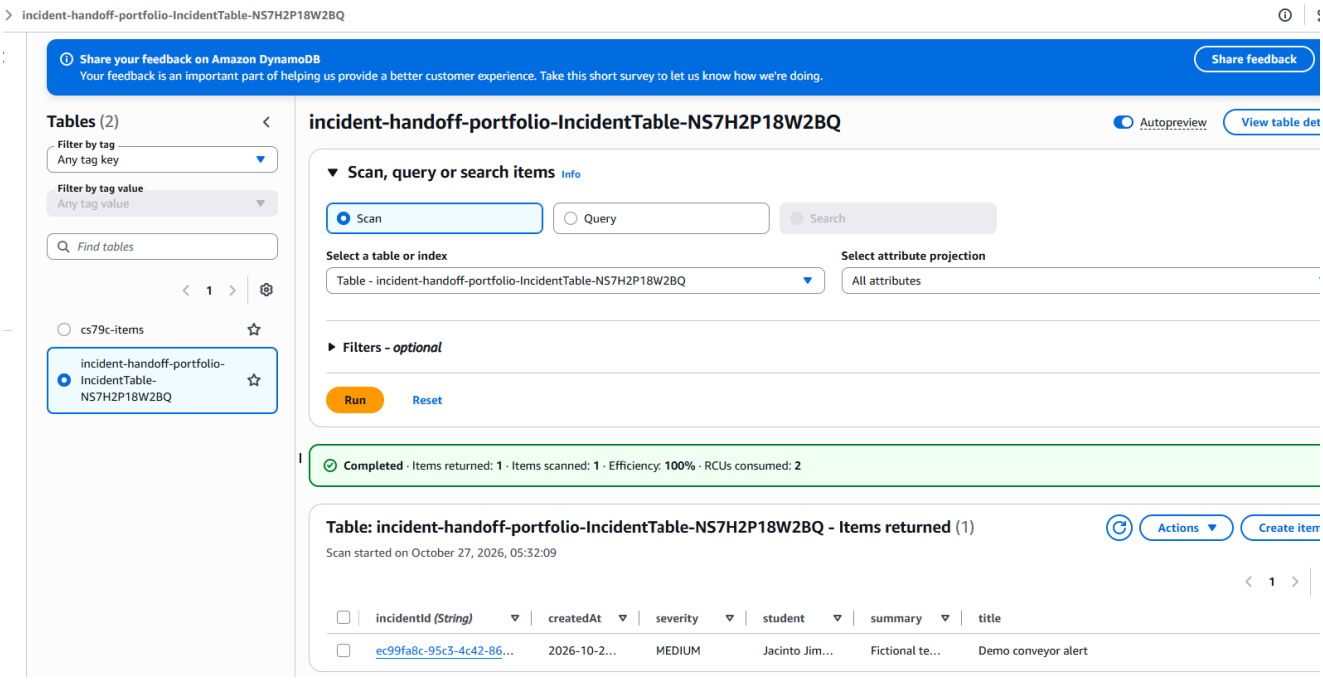
Live HTTPS API Gateway /health returned project, author and status ok.

Screenshot 3 — Lambda POST Incident



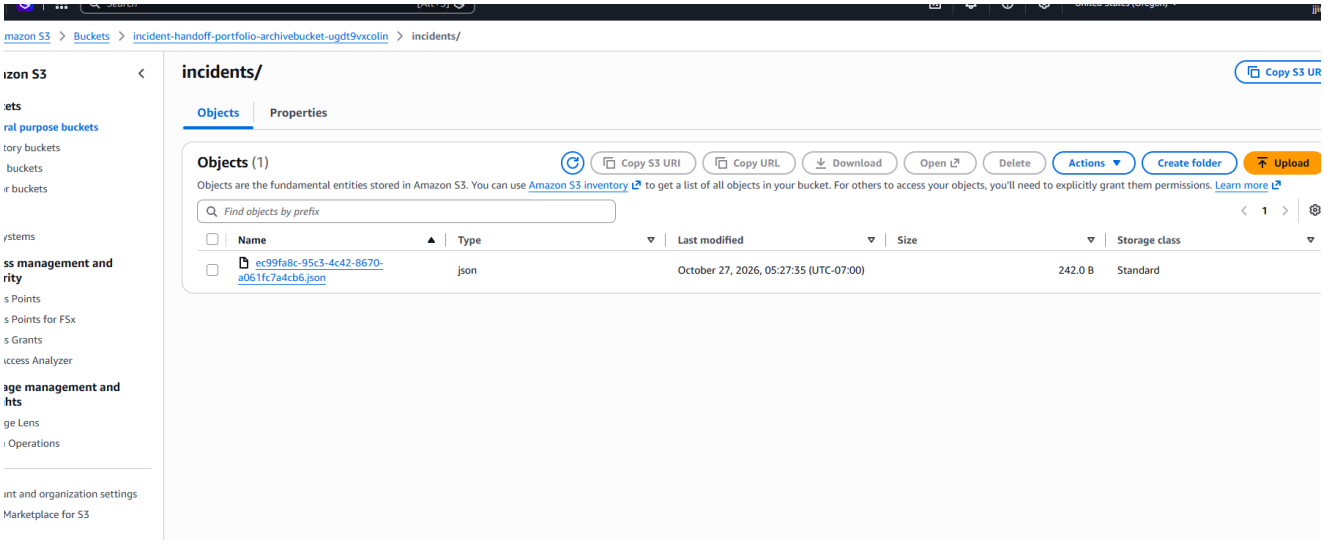
Direct Lambda console event returned statusCode 201 for a fictional incident.

Screenshot 4 — DynamoDB Persistence



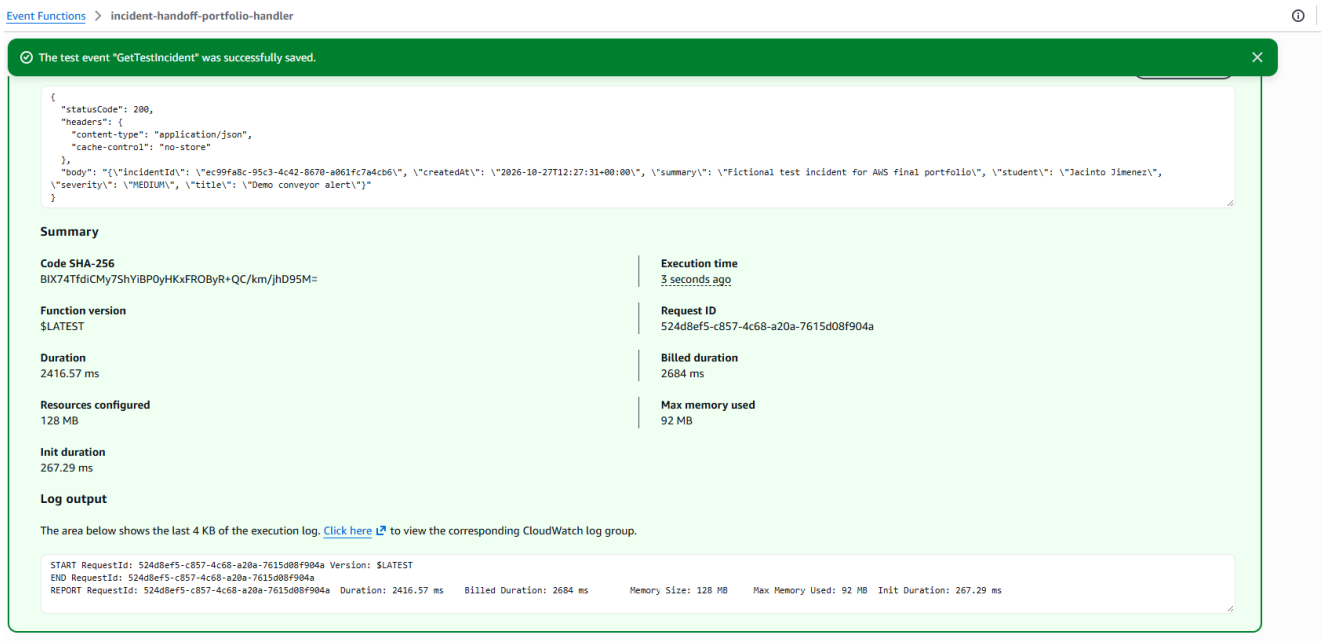
DynamoDB Scan returned the incident matching the Lambda response.

Screenshot 5 — Amazon S3 JSON Archive



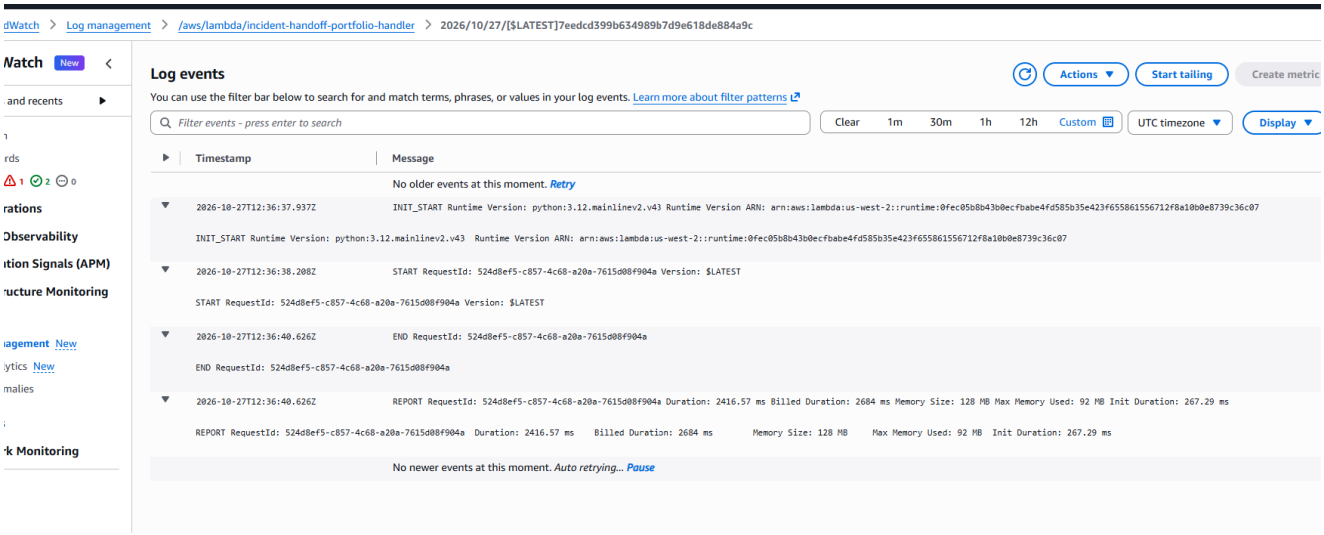
Private S3 incidents/ prefix contained the matching UUID .json object.

Screenshot 6 — Lambda GET Incident



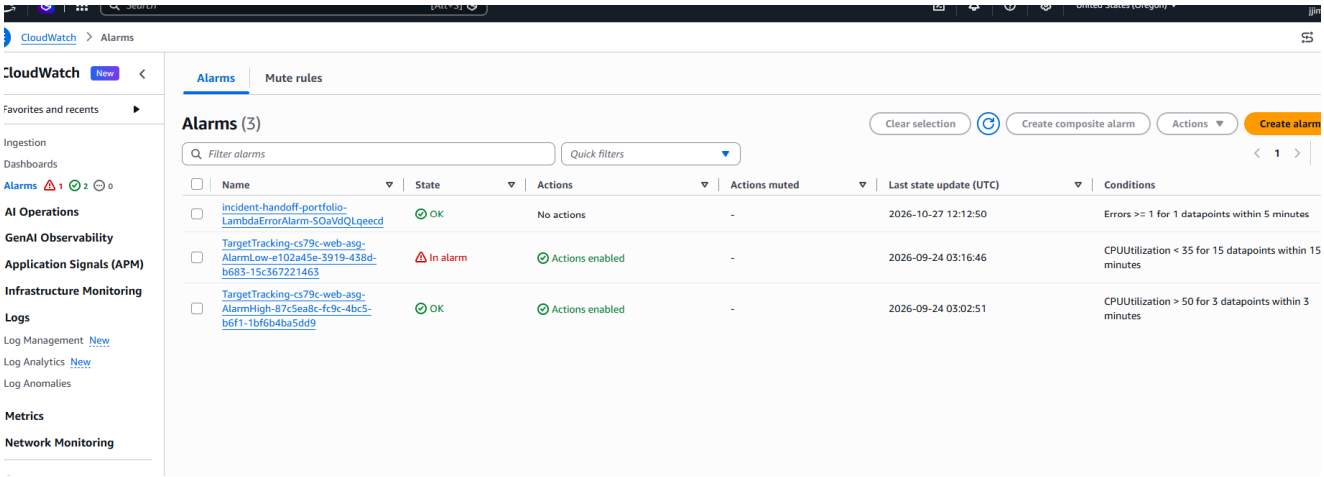
Direct Lambda console event returned statusCode 200 and the matching record.

Screenshot 7 — CloudWatch Execution Logs



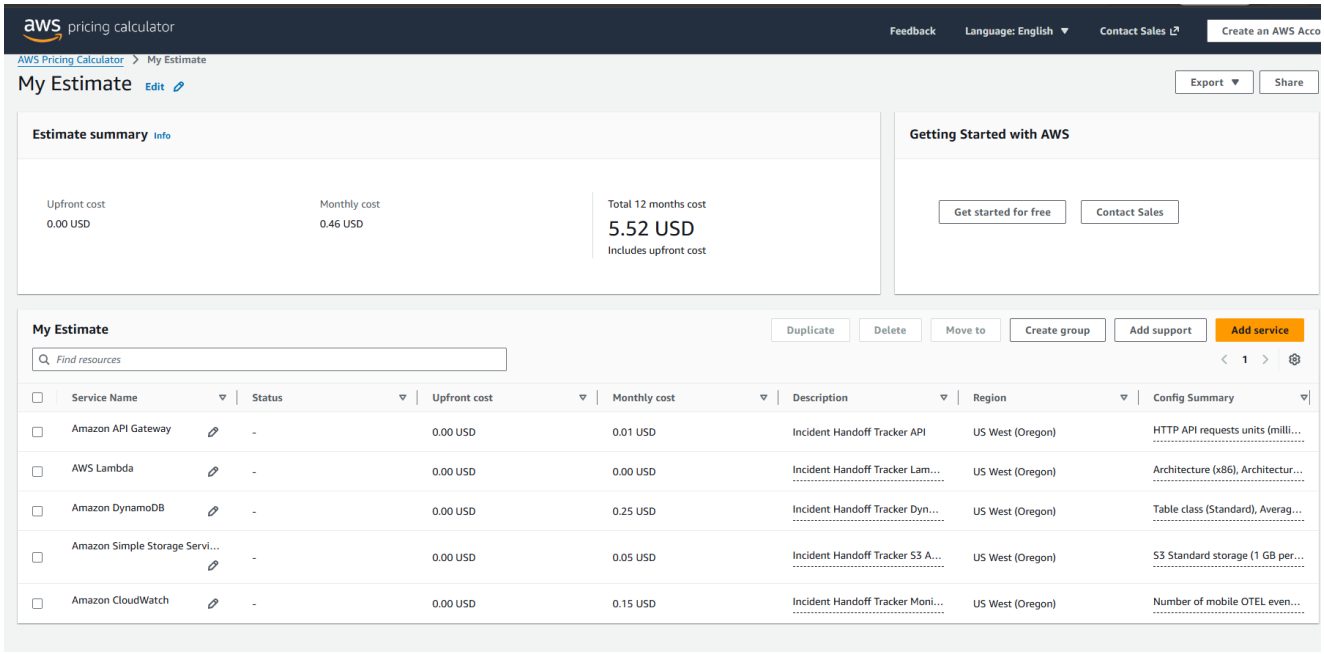
START/END/REPORT entries document the GET execution and resource usage.

Screenshot 8 — CloudWatch Lambda Error Alarm



The portfolio Lambda error alarm existed and was in the OK state.

Screenshot 9 — AWS Pricing Calculator



Five-service hypothetical estimate was $0.46/month and $5.52/year.

References

AWS CloudFormation template documentation: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-guide.html

API Gateway HTTP API IAM authorization: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-access-control-iam.html

AWS Well-Architected Framework: https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html

AWS Pricing Calculator: https://calculator.aws/